Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 7: Class & Object (OOP-1)

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. OOP & Objects
2. Class & Object
3. Constructor (__init__)
4. Method vs Function
5. Magic / Dunder Methods
6. When to Use a Constructor
7. Concept of self

42 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 7: Class & Object (OOP-1) - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: OOP & Objects, Class & Object, Constructor (__init__), Method vs Function, Magic / Dunder Methods, When to Use a Constructor, Concept of self
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. OOP & Objects

Q1. Everything is an Object

BEGINNER

Print the type of 10, "Pune", [1, 2] and {"a": 1}. Notice that each one is an object of a built-in class.

Solution:

```
print(type(10))
print(type("Pune"))
print(type([1, 2]))
print(type({"a": 1}))
```

Output:

```
<class 'int'>
<class 'str'>
<class 'list'>
<class 'dict'>
```

Q2. Objects Have Methods

BEGINNER

The string "bikkad it" is an object. Call its upper() and title() methods, and use isinstance() to check it is a str object.

Solution:

```
name = "bikkad it"

print(name.upper())
print(name.title())
print(isinstance(name, str))
```

Output:

```
BIKKAD IT
Bikkad It
True
```

Q3. Without OOP vs With OOP

INTERMEDIATE

Store two cars without classes (separate variables) and then with a simple class Car having brand and price. Print both cars using the class.

Solution:

```
# Without OOP - many loose variables
car1_brand = "Tata"
car1_price = 800000

# With OOP - data kept together inside objects
class Car:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

c1 = Car("Tata", 800000)
c2 = Car("Mahindra", 1200000)
print(c1.brand, c1.price)
print(c2.brand, c2.price)
```

Output:

```
Tata 800000
Mahindra 1200000
```

Q4. Identify State and Behaviour

INTERMEDIATE

A bank account has state (holder, balance) and behaviour (deposit). Write class Account with these and deposit 2000 into an account with 5000.

Solution:

```
class Account:
    def __init__(self, holder, balance):
        self.holder = holder        # state
        self.balance = balance      # state

    def deposit(self, amount):      # behaviour
        self.balance += amount

acc = Account("Meera", 5000)
acc.deposit(2000)
print(acc.balance)
```

Output:

```
7000
```

Q5. Each Object is Independent

ADVANCED

Create two Mobile objects with battery 100. Use one for 30% and the other for 55%. Show that the batteries change independently.

Solution:

```
class Mobile:
    def __init__(self, owner):
        self.owner = owner
        self.battery = 100

    def use(self, percent):
        self.battery -= percent

m1 = Mobile("Om")
m2 = Mobile("Pia")
m1.use(30)
m2.use(55)
print(m1.owner, m1.battery)
print(m2.owner, m2.battery)
```

Output:

```
Om 70
Pia 45
```

Q6. Model a Real-World Object

ADVANCED

Model a LibraryBook with title, author, is_issued (False by default) and methods issue() and return_book() that print a message and update the status.

Solution:

```
class LibraryBook:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_issued = False

    def issue(self):
        if self.is_issued:
            print(self.title, "is already issued")
        else:
            self.is_issued = True
            print(self.title, "issued")

    def return_book(self):
        self.is_issued = False
        print(self.title, "returned")

b = LibraryBook("Wings of Fire", "Kalam")
b.issue()
b.issue()
b.return_book()
```

Output:

```
Wings of Fire issued
Wings of Fire is already issued
Wings of Fire returned
```

2. Class & Object

Q7. Create an Empty Class

BEGINNER

Create a class Student with pass. Create an object s1 and print isinstance(s1, Student).

Solution:

```
class Student:
    pass

s1 = Student()
print(isinstance(s1, Student))
print(type(s1).__name__)
```

Output:

```
True
Student
```

Q8. Class with a Method

BEGINNER

Create class Cafe with a method open_shop() that prints "Cafe is open". Create an object and call the method.

Solution:

```
class Cafe:
    def open_shop(self):
        print("Cafe is open")

c = Cafe()
c.open_shop()
```

Output:

```
Cafe is open
```

Q9. Multiple Objects

INTERMEDIATE

Create class Product with name and price. Create 3 products and print the name and price of each.

Solution:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

p1 = Product("Pen", 10)
p2 = Product("Book", 60)
p3 = Product("Bag", 450)

for p in [p1, p2, p3]:
    print(p.name, p.price)
```

Output:

```
Pen 10
Book 60
Bag 450
```

Q10. Method that Returns a Value

INTERMEDIATE

Create class Rectangle with length and width and methods area() and perimeter(). Test with 12 x 8.

Solution:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r = Rectangle(12, 8)
print(r.area())
print(r.perimeter())
```

Output:

```
96
40
```

Q11. Shopping Cart Class

ADVANCED

Create class Cart with an empty items list. Add methods add(name, price), total() and count(). Add 3 items and print the total and count.

Solution:

```
class Cart:
    def __init__(self):
        self.items = []

    def add(self, name, price):
        self.items.append((name, price))

    def total(self):
        return sum(price for name, price in self.items)

    def count(self):
        return len(self.items)

cart = Cart()
cart.add("Shoes", 1999)
cart.add("Socks", 199)
cart.add("Cap", 349)
print(cart.total())
print(cart.count())
```

Output:

```
2547
3
```

Q12. Student Report Card

ADVANCED

Create class Student with name and a marks dictionary. Add methods percentage() and grade() (A >= 75, B >= 60, C >= 35, else F).

Solution:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def percentage(self):
        total = sum(self.marks.values())
        return round(total / len(self.marks), 2)

    def grade(self):
        p = self.percentage()
        if p >= 75:
            return "A"
        elif p >= 60:
            return "B"
        elif p >= 35:
            return "C"
        return "F"

s = Student("Kavya", {"Maths": 82, "Science": 74, "English": 69})
print(s.percentage())
print(s.grade())
```

Output:

```
75.0
A
```

3. Constructor (__init__)

Q13. Constructor Runs Automatically

BEGINNER

Create class Order whose constructor prints "Order created". Create two objects and see it run each time.

Solution:

```
class Order:
    def __init__(self):
        print("Order created")

o1 = Order()
o2 = Order()
```

Output:

```
Order created
Order created
```

Q14. Constructor with Parameters

BEGINNER

Create class Employee whose constructor takes name and salary and stores them. Print both for one employee.

Solution:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

e = Employee("Ajay", 45000)
print(e.name)
print(e.salary)
```

Output:

```
Ajay
45000
```

Q15. Default Values in Constructor

INTERMEDIATE

Create class Pizza(size="Medium", cheese=False). Create one pizza with defaults and one Large pizza with cheese.

Solution:

```
class Pizza:
    def __init__(self, size="Medium", cheese=False):
        self.size = size
        self.cheese = cheese

p1 = Pizza()
p2 = Pizza("Large", True)
print(p1.size, p1.cheese)
print(p2.size, p2.cheese)
```

Output:

```
Medium False
Large True
```

Q16. Calculated Attribute

INTERMEDIATE

Create class Invoice(amount, gst_rate) whose constructor also calculates self.total = amount + gst. Print the total for 2000 at 18%.

Solution:

```
class Invoice:
    def __init__(self, amount, gst_rate):
        self.amount = amount
        self.gst_rate = gst_rate
        self.total = amount + amount * gst_rate / 100

inv = Invoice(2000, 18)
print(inv.total)
```

Output:

```
2360.0
```

Q17. Validation in Constructor

ADVANCED

Create class BankAccount(holder, balance). If balance is negative, set it to 0 and print a warning.

Solution:

```
class BankAccount:
    def __init__(self, holder, balance):
        self.holder = holder
        if balance < 0:
            print("Warning: negative balance set to 0")
            balance = 0
        self.balance = balance

a1 = BankAccount("Om", 5000)
a2 = BankAccount("Pia", -200)
print(a1.balance)
print(a2.balance)
```

Output:

```
Warning: negative balance set to 0
5000
0
```

Q18. Auto Generated ID

ADVANCED

Create class Ticket where every new object gets a ticket number automatically (T1, T2, T3...) using a class variable counter inside the constructor.

Solution:

```
class Ticket:
    counter = 0

    def __init__(self, passenger):
        Ticket.counter += 1
        self.ticket_no = "T" + str(Ticket.counter)
        self.passenger = passenger

t1 = Ticket("Om")
t2 = Ticket("Pia")
t3 = Ticket("Raj")
print(t1.ticket_no, t1.passenger)
print(t3.ticket_no, t3.passenger)
```

Output:

```
T1 Om
T3 Raj
```

4. Method vs Function

Q19. Function vs Method

BEGINNER

Write a normal function add(a, b). Also call the list method append() on a list. Explain in comments which is a function and which is a method.

Solution:

```
def add(a, b):          # function - not inside a class
    return a + b

print(add(3, 4))

nums = [1, 2]
nums.append(3)          # method - belongs to the list object
print(nums)
```

Output:

```
7
[1, 2, 3]
```

Q20. Built-in Function vs Method

BEGINNER

Use len("Python") (function) and "Python".upper() (method). Print both.

Solution:

```
word = "Python"

print(len(word))      # function
print(word.upper())   # method of the str object
```

Output:

```
6
PYTHON
```

Q21. Same Logic as Function and Method

INTERMEDIATE

Calculate the area of a circle using a function circle_area(r) and also using a method area() inside class Circle.

Solution:

```
import math

def circle_area(r):
    return round(math.pi * r * r, 2)

class Circle:
    def __init__(self, r):
        self.r = r

    def area(self):
        return round(math.pi * self.r * self.r, 2)

print(circle_area(5))
print(Circle(5).area())
```

Output:

```
78.54
78.54
```

Q22. Calling a Method Through the Class

INTERMEDIATE

For class Greeter with method hello(self), call it as obj.hello() and as Greeter.hello(obj). Both give the same result.

Solution:

```
class Greeter:
    def __init__(self, name):
        self.name = name

    def hello(self):
        print("Hello", self.name)

g = Greeter("Isha")
g.hello()
Greeter.hello(g)
```

Output:

```
Hello Isha
Hello Isha
```

Q23. Methods Change Object State

ADVANCED

Create class Counter with methods increment() and reset(). A function cannot remember a value between calls without globals, but a method can. Show it.

Solution:

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

    def reset(self):
        self.value = 0

c = Counter()
c.increment()
c.increment()
print(c.value)
c.reset()
print(c.value)
```

Output:

```
2
0
```

Q24. Bound Method Object

ADVANCED

Store a bound method acc.deposit in a variable and call it twice. Print acc.balance.

Solution:

```
class Account:
    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount

acc = Account()
dep = acc.deposit       # bound method remembers acc
dep(500)
dep(700)
print(acc.balance)
```

Output:

```
1200
```

5. Magic / Dunder Methods

Q25. __str__ for Nice Printing

BEGINNER

Create class Book(title, price) with __str__ returning "Title - Rs.price". Print a book object.

Solution:

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __str__(self):
        return f"{self.title} - Rs.{self.price}"

b = Book("Python Basics", 450)
print(b)
```

Output:

```
Python Basics - Rs.450
```

Q26. __len__

BEGINNER

Create class Playlist with a list of songs and __len__ returning the number of songs. Use len() on the object.

Solution:

```python
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __len__(self):
        return len(self.songs)

p = Playlist(["Song A", "Song B", "Song C"])
print(len(p))
```

Output:

```
3
```

Q27. __add__ for Wallets

INTERMEDIATE

Create class Wallet(amount) where adding two wallets with + returns a new Wallet with the total. Add __str__ too.

Solution:

```python
class Wallet:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Wallet(self.amount + other.amount)

    def __str__(self):
        return f"Wallet(Rs.{self.amount})"

w = Wallet(500) + Wallet(750)
print(w)
```

Output:

```
Wallet(Rs.1250)
```

Q28. __eq__ to Compare Objects

INTERMEDIATE

Create class Point(x, y) with __eq__ so that two points with the same x and y are equal.

Solution:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

print(Point(2, 3) == Point(2, 3))
print(Point(2, 3) == Point(3, 2))
```

Output:

```
True
False
```

Q29. __lt__ to Sort Objects

ADVANCED

Create class Employee(name, salary) with __lt__ comparing salary and __repr__. Sort a list of 3 employees.

Solution:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __lt__(self, other):
        return self.salary < other.salary

    def __repr__(self):
        return f"{self.name}({self.salary})"

staff = [Employee("Om", 52000),
         Employee("Pia", 39000),
         Employee("Raj", 61000)]
print(sorted(staff))
```

Output:

```
[Pia(39000), Om(52000), Raj(61000)]
```

Q30. __getitem__ and __contains__

ADVANCED

Create class Classroom with a list of students. Support classroom[0] using __getitem__ and "Pia" in classroom using __contains__.

Solution:

```
class Classroom:
    def __init__(self, students):
        self.students = students

    def __getitem__(self, index):
        return self.students[index]

    def __contains__(self, name):
        return name in self.students

c = Classroom(["Om", "Pia", "Raj"])
print(c[0])
print(c[-1])
print("Pia" in c)
print("Zoe" in c)
```

Output:

```
Om
Raj
True
False
```

6. When to Use a Constructor

Q31. Required Data at Creation

BEGINNER

A Student must always have a roll number and name. Use a constructor so they are set when the object is created.

Solution:

```
class Student:
    def __init__(self, roll, name):
        self.roll = roll
        self.name = name

s = Student(21, "Kavya")
print(s.roll, s.name)
```

Output:

```
21 Kavya
```

Q32. Setting Up Defaults

BEGINNER

A new GameScore object should always start with score 0 and lives 3. Set these in the constructor.

Solution:

```
class GameScore:
    def __init__(self, player):
        self.player = player
        self.score = 0
        self.lives = 3

g = GameScore("Om")
print(g.player, g.score, g.lives)
```

Output:

```
Om 0 3
```

Q33. Without Constructor Problem

INTERMEDIATE

Create class Car without a constructor and try to print car.brand before setting it, inside try-except. Then fix it using a constructor.

Solution:

```
class CarNoInit:
    pass

c = CarNoInit()
try:
    print(c.brand)
except AttributeError as e:
    print(e)

class Car:
    def __init__(self, brand):
        self.brand = brand

print(Car("Tata").brand)
```

Output:

```
'CarNoInit' object has no attribute 'brand'
Tata
```

Q34. Prepare an Empty Container

INTERMEDIATE

A Basket class should start with an empty list of fruits for EVERY new object. Create two baskets and add fruit to only one.

Solution:

```
class Basket:
    def __init__(self):
        self.fruits = []

b1 = Basket()
b2 = Basket()
b1.fruits.append("Mango")
print(b1.fruits)
print(b2.fruits)
```

Output:

```
['Mango']
[]
```

Q35. Constructor Calls Another Method

ADVANCED

Create class Order(items) where the constructor stores items and calls self.calculate_total() to set self.total automatically.

Solution:

```
class Order:
    def __init__(self, items):
        self.items = items
        self.total = self.calculate_total()

    def calculate_total(self):
        return sum(qty * price for qty, price in self.items)

o = Order([(2, 120), (1, 450)])
print(o.total)
```

Output:

```
690
```

Q36. Connection Setup Simulation

ADVANCED

Create class Database(name) whose constructor prints "Connecting to <name>" and sets connected = True. Add a close() method that sets it to False.

Solution:

```
class Database:
    def __init__(self, name):
        self.name = name
        print("Connecting to", name)
        self.connected = True

    def close(self):
        self.connected = False
        print("Closed", self.name)

db = Database("students_db")
print(db.connected)
db.close()
print(db.connected)
```

Output:

```
Connecting to students_db
True
Closed students_db
False
```

7. Concept of self

Q37. self Refers to the Object

BEGINNER

Create class Dog with a method show() that prints self.name. Create two dogs and call show() on each.

Solution:

```
class Dog:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("I am", self.name)

d1 = Dog("Tommy")
d2 = Dog("Bruno")
d1.show()
d2.show()
```

Output:

```
I am Tommy
I am Bruno
```

Q38. self is the Same Object

BEGINNER

Inside a method, return id(self). Compare it with id(obj) to prove self is the object itself.

Solution:

```
class Box:
    def get_id(self):
        return id(self)

b = Box()
print(b.get_id() == id(b))
```

Output:

```
True
```

Q39. Forgetting self

INTERMEDIATE

Define a method without self and call it on an object inside try-except to see the error.

Solution:

```
class Bad:
    def hello():
        print("hello")

obj = Bad()
try:
    obj.hello()
except TypeError as e:
    print(e)
```

Output:

```
Bad.hello() takes 0 positional arguments but 1 was given
```

Q40. Method Calls Another Method

INTERMEDIATE

Create class Thermostat with to_fahrenheit() and a report() method that uses self.to_fahrenheit().

Solution:

```
class Thermostat:
    def __init__(self, celsius):
        self.celsius = celsius

    def to_fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    def report(self):
        print(self.celsius, "C =", self.to_fahrenheit(), "F")

Thermostat(30).report()
```

Output:

```
30 C = 86.0 F
```

Q41. Method Chaining with return self

ADVANCED

Create class Query with methods where(), order_by() that store text and return self so you can chain them:
Query("users").where("age>18").order_by("name").

Solution:

```
class Query:
    def __init__(self, table):
        self.text = "SELECT * FROM " + table

    def where(self, condition):
        self.text += " WHERE " + condition
        return self

    def order_by(self, column):
        self.text += " ORDER BY " + column
        return self

q = Query("users").where("age > 18").order_by("name")
print(q.text)
```

Output:

```
SELECT * FROM users WHERE age > 18 ORDER BY name
```

Q42. self Name is Only a Convention

ADVANCED

Write a class where the first parameter is named this instead of self. Show that it still works (but explain in a comment why self is recommended).

Solution:

```
class Person:
    def __init__(this, name):
        this.name = name

    def greet(this):
        print("Hi", this.name)

Person("Neha").greet()
# Python only needs a first parameter; "self" is the
# standard name every Python developer expects, so use it.
```

Output:

```
Hi Neha
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026